SEGURIDAD

Bases de Datos 2026

MOTIVACIÓN

Una base de datos real maneja información sensible, que debe protegerse de accesos y modificaciones malintencionadas.

Contraseñas y tarjetas **Historia clínica** **Pedidos de un cliente**

- Cronología de Ciberincidentes en Argentina | Cumplimiento Digital + Ciberseguridad Argentina

SISTEMA DE SEGURIDAD DEL DBMS

Confidencialidad	Integridad	Disponibilidad
Protección de los datos frente a su revelación no autorizada.	Solo usuarios autorizados pueden modificar los datos.	Los datos deben estar accesibles para usuarios y programas autorizados.

Protección dirigida a dos tipos de usuarios:

- Los que no tienen derecho de acceso
- Los que tienen derecho limitado a ciertas acciones

¿QUÉ SE PUEDE PROTEGER?

La granularidad de la protección puede variar en escala.

- Un atributo de una tupla: una sola columna, de una sola fila.
- Un conjunto de columnas: por ejemplo, las columnas sensibles de una tabla.
- Una tupla individual: una fila completa.
- Un conjunto de tuplas de una relación: varias filas, por ejemplo, todas las que cumplen una condición.
- Una relación en particular: una tabla entera.
- Un conjunto de relaciones: varias tablas.
- La base de datos completa.

MÉTODOS PARA EL CONTROL DE ACCESOS

Control de Acceso Discrecional

- Garantiza privilegios a usuarios, incluyendo la capacidad para acceder archivos de datos específicos, registros o campos para operar de una manera determinada (select, insert, delete, o update).

SEGURIDAD A NIVEL DBMS

Encriptado

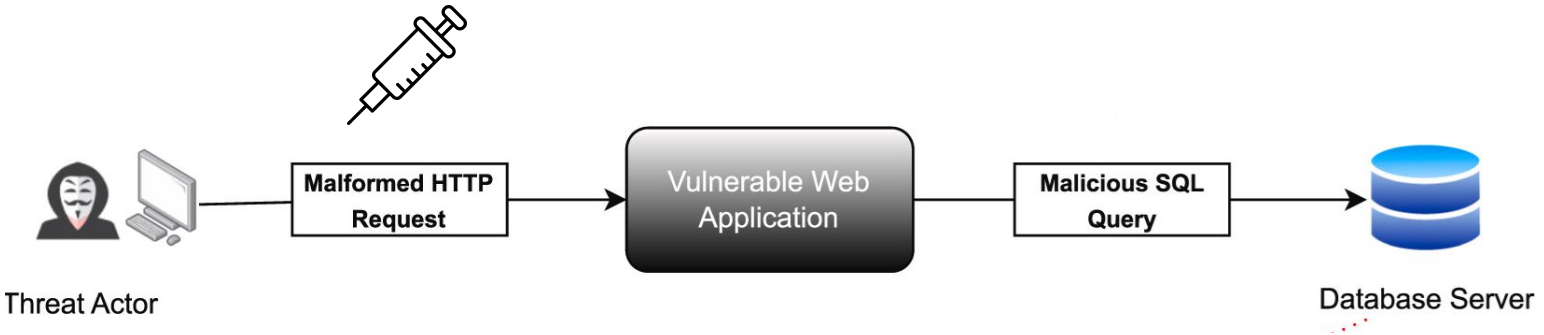
Los datos son ilegibles sin la clave. Protege backups, discos robados y credenciales

Audit trail

Registra quién accedió a los datos y qué operaciones realizó

ATAQUE POR INYECCIÓN SQL

- Ocurre cuando algunos datos del usuario están destinados a modificar una sentencia SQL
- Vulnerabilidades reportadas (2005-Actualidad)
  - Common Vulnerabilities and Exposures
- Casos conocidos de SQL Injection (SQLi)
  - 2015 TalkTalk data breach - Wikipedia
  - SQL injection vulnerability in MOVEit Transfer (2023)
- El input de un usuario que puede modificar una sentencia SQL debe ser cuidadosamente editado para asegurar que sólo se reciben inputs válidos y que no se ha ingresado código adicional



Paul, A., Sharma, V. & Olukoya, O. (2024). SQL injection attack: Detection, prioritization & prevention. Journal of Information Security and Applications, Volume 85,103871,ISSN 2214-2126. https://www.sciencedirect.com/science/article/pii/S221421262400173X?via%3Dihub.

ATAQUE POR INYECCIÓN SQL - Ejemplos

- Ejemplo: en un campo de texto de un formulario en la WEB se requiere a los usuarios que ingresen su nombre.

```
SELECT * FROM instructor
WHERE name = X;
```

- El usuario ingresa: `'Juan Durán' OR TRUE`

```
SELECT * FROM instructor
WHERE name = 'Juan Durán' OR TRUE;
```

Se **obtiene** toda la **tabla instructor**

ATAQUE POR INYECCIÓN SQL - Ejemplos

```
SELECT * FROM instructor
WHERE name = 'X';
```

- El usuario ingresa: `a'; drop table instructor;--`

```
SELECT * FROM instructor
WHERE name = 'a'; DROP TABLE instructor;--';
```

```
SELECT * FROM instructor
WHERE name = 'a'; DROP TABLE instructor;--';
```

La tabla instructor **es eliminada**

¿CÓMO PREVENIR LA INYECCIÓN SQL?

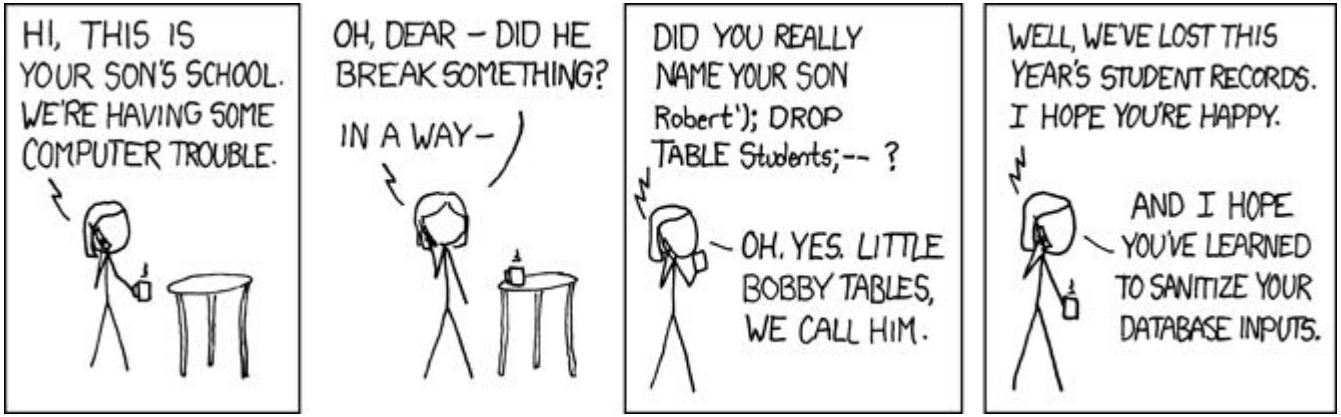
> *Creating SQL queries by concatenating strings with user-input values is an extremely serious security risk and should never be done in any program.*



Prepared Statements (Queries parametrizadas)

Agregan automáticamente caracteres de escape a los parámetros, evitando que el input cierre o altere la consulta.

🔗 IT'S DEMO TIME



Good luck speed cameras



SEGURIDAD A NIVEL USUARIO

- Cada usuario tiene ciertos derechos sobre ciertos objetos.
- Distintos usuarios pueden tener los mismos o distintos derechos sobre los mismos objetos.
- Para controlar la granularidad de los derechos de acceso, los usuarios pueden tener derechos (autorización / privilegios) sobre
    - Tablas
    - Vistas

CONTROL DE ACCESO DISCRECIONAL

- En SQL standard se incluyen los privilegios sobre tablas o vistas
  - **ALL, CREATE, SELECT, INSERT, UPDATE, DELETE.**
- En el lenguaje de definición de SQL se incluyen los comandos **GRANT** y **REVOKE** para administrar privilegios.
  - Solo ciertos usuarios o roles tienen privilegios sobre datos confidenciales.

GRANT Concede autorización

```
GRANT <privilegios>
ON <tabla o vista>
TO <usuarios / roles>;
```

REVOKE Anula autorización

```
REVOKE <privilegios>
ON <tabla o vista>
FROM <usuarios / roles>;
```

GRANT - Ejemplo

```
GRANT SELECT
ON department
TO Mafalda, Guille;
```

```
GRANT UPDATE (budget)
ON department
TO Mafalda, Guille;
```

REVOKE - Ejemplo

```
REVOKE SELECT
ON department
FROM Mafalda, Guille;
```

```
REVOKE UPDATE (budget)
ON department
FROM Mafalda, Guille;
```

ROLES

- Problema
  - Si hay muchos usuarios con muchos privilegios diferentes, resultará difícil adicionar nuevos privilegios a cada individuo.
- Surge la idea de **Roles**
  - Grupos de privilegios relacionados que se otorgan a usuarios.
  - Si se cambian los privilegios encapsulados en un rol, los privilegios de todos los usuarios que tienen ese rol también cambian.

ROLES - Ejemplo

```
-- Creación
CREATE ROLE instructor;

-- Asignación de privilegios a roles
GRANT select ON takes TO instructor;

-- Asignación de privilegios de roles a roles y a
usuarios
CREATE ROLE dean;
GRANT instructor TO dean;
GRANT dean TO Mafalda;
```

Algunos consejos de seguridad

1. Proteger las cuentas privilegiadas (como root) con contraseñas fuertes.
2. Únicamente el usuario root debe tener acceso a la tabla **mysql.user**
3. No dar más privilegios que los necesarios
4. No dar permisos a todos los hosts (comodín '%')
5. No almacenar contraseñas (como datos) en texto plano
   - Sony Data Breach (2011)